Verifier-Guided Remasking for Robust Masked Diffusion in Logical Reasoning

Martin Eigel
Weierstrass Institute, Berlin
eigel@wias-berlin.de

June 1, 2026

Abstract

Masked diffusion models generate sequences by iterative denoising, making them natural candidates for globally constrained generation tasks such as program repair, theorem proving, and structured reasoning. However, standard masked diffusion assumes that observed tokens are always correct. When a state contains wrong tokens — whether from an imperfect base model, previous erroneous predictions, or noisy observations — the denoising process has no mechanism to correct them. We propose **verifier-guided remasking**, a simple modification that closes this gap. At each diffusion step, a symbolic verifier identifies inconsistent positions via local constraint residuals $r_i(x)$; these positions are temporarily returned to the masked state, allowing the denoiser to propose corrections. On 4×4 Sudoku with up to 30% wrong tokens, verifier-guided remasking achieves 98–100% completion versus 8–14% for standard masked diffusion. On planted 3-SAT with 50 different random formulas, the repair mechanism maintains 74–86% success versus 18–26% without repair. Crucially, a control that uses verifier-guided remasking but fills with a random denoiser achieves 0% — showing in this setting that the repair mechanism alone is insufficient without a structured denoiser. The framework applies to any domain with a verifier that produces localizable constraint residuals: we demonstrate it on both Sudoku (all-different constraints) and SAT (clause satisfaction).

1 Introduction

Masked diffusion language models (MDLMs) [1–3] generate sequences by iterative denoising: starting from a partially masked state, a model predicts clean tokens at masked positions, and a policy selects which positions to unmask at each step. Unlike autoregressive models, MDLMs can refine arbitrary positions using bidirectional context, making them attractive for tasks requiring global consistency — code generation, theorem proving, structured reasoning, and constraint satisfaction.

A fundamental limitation of standard masked diffusion is its assumption that *observed tokens are correct*. Once a position is unmasked, its value is fixed unless the policy explicitly supports remasking. When the state contains wrong tokens — due to model error, noisy inputs, or previous incorrect predictions — the diffusion process has no mechanism to detect or correct them. As we show experimentally, this causes success rates to collapse from 100% to below 15% as the fraction of wrong tokens increases from 0 to 30%.

We propose a simple remedy: **verifier-guided remasking**. At each diffusion step, a domain-specific symbolic verifier computes:

- A global violation score $V(x) = \sum_{G \in \mathcal{G}} \mathbf{1}\{\text{group } G \text{ is violated}\}$, and
- Local residuals $r_i(x) = |\{G \in \mathcal{G} : i \in G \text{ and } G \text{ is violated}\}|$,

where $\mathcal{G}$ is the set of constraint groups (e.g. rows, columns, boxes for Sudoku; clauses for SAT). Positions with $r_i(x) > 0$ are candidates for remasking: they are returned to `MASK`, and the denoiser proposes new values on the next step.

This mechanism applies to any domain with a verifier that produces localizable constraint residuals. We evaluate it on two structurally different domains: 4×4 Sudoku (all-different constraints over 16 variables, 12 groups) and planted 3-SAT (random 20-variable formulas with 60 clause factors). In both cases, verifier-guided remasking dramatically improves robustness to wrong tokens.

Our key contributions are:

- Identifying a fundamental failure mode of masked diffusion: wrong observed tokens cause irreversible contamination.
- Proposing verifier-guided remasking as a domain-agnostic remedy, requiring only a verifier producing local residuals.
- Demonstrating 98–100% repair success on Sudoku and 74–86% on SAT under high wrong-token ratios, versus 8–26% without repair.
- Introducing mechanism-isolating controls showing that verifier remasking alone is insufficient: success requires a denoiser with useful constraint information.
- Validating on 50 different random SAT formulas to ensure results generalize beyond a single instance.

2 Related work

Masked diffusion models. MDLMs [1–3] generate discrete sequences by iterative unmasking. The mask policy — which positions to fill at each step — is an active research topic [6]. Our contribution is orthogonal: we augment the policy with verifier feedback, allowing it to repair observed positions that violate constraints.

Neuro-symbolic reasoning. Combining neural networks with symbolic verifiers has a long history [7, 8]. Our approach is closest to inference-time correction, where a verifier checks outputs and triggers repair. Unlike methods that train with logical losses, our verifier is purely symbolic and requires no gradient propagation.

Tensor-network inference. Tensor networks provide compact factor-graph inference for discrete CSPs [4, 5]. We use exact contraction (variable elimination via einsum) to compute logical marginals, which serve as the denoising distribution. The key finding is that the verifier—not the marginal computation—is the decisive component for repair.

3 Method

3.1 Masked diffusion with repair

Let $x \in (A \cup \{\texttt{MASK}\})^n$ be a masked state over finite domain A of size d. The standard diffusion step is:

1. Denoise: $p \leftarrow \text{Denoiser}(x)$.
2. Select: $U \leftarrow \text{Policy}(x, p)$ (positions to unmask).
3. Fill: $x_i \leftarrow \text{argmax}_v\, p_i(v)$ for $i \in U$.

Our repair-augmented step adds one phase before denoising:

1. **Verify:** $d \leftarrow \text{Verifier}(x)$.

2. **Remask:** $x_i \leftarrow \texttt{MASK}$ for i where $\text{Policy}_{\text{remask}}(x, d) = \text{True}$.

3. Denoise: $p \leftarrow \text{Denoiser}(x)$.

4. Fill: $x_i \leftarrow \text{argmax}_v\, p_i(v)$ for $i \in \text{Policy}_{\text{fill}}(x, p, d)$.

The remask policy selects observed positions with positive local residual $r_i(x) > 0$. The fill policy uses confidence thresholding (unmask if $\max_v p_i(v) \geq 0.99$; guarantee at least one fill per step).

3.2 Verifier

The verifier checks constraint satisfaction. For Sudoku, each variable belongs to three all-different groups (one row, one column, one 2×2 box). A group is violated if any two observed variables share the same value:

$$V(x) = |\{G \in \mathcal{G} : \exists i, j \in G,\ i \neq j,\ x_i = x_j \neq \texttt{MASK}\}|. \tag{1}$$

For SAT, each clause $C = (\ell_1 \vee \cdots \vee \ell_k)$ is a factor with table $\psi_C(x_{\partial C}) \in \{0, 1\}$ indicating whether at least one literal is true. A clause is violated iff all observed literals are false and no variables are masked.

The local residual $r_i(x)$ counts violated groups containing i.

3.3 Tensor-factor backend

Given observed positions Ω, the denoiser computes exact conditional marginals:

$$q_i(v) = P_\Psi(x_i = v \mid x_\Omega) = \frac{\sum_{x_{M\setminus\{i\}}} \Psi(x_\Omega, x_i = v, x_{M\setminus\{i\}})}{\sum_{x_M} \Psi(x_\Omega, x_M)}, \tag{2}$$

where $\Psi(x) = \prod_{G\in\mathcal{G}} \psi_G(x_G)$ is the product of constraint factors. We compute these marginals via variable elimination (joining factors by einsum and sequentially eliminating non-target variables). For Sudoku (288 solutions, 12 factors of arity 4), this is exact to machine precision ($\|q^{\text{VE}} - q^{\text{exact}}\|_\infty < 10^{-14}$).

3.4 Denoisers

We compare several denoisers:

- **Random:** $p_i(v) = 1/d$ (baseline).
- **Local heuristic:** uniform over values not locally forbidden by observed neighbours in shared constraint groups.
- **TN marginal:** $p_i(v) = q_i(v)$ from the contraction backend.
- **PoE:** $\log \tilde{p}_i(v) = \beta \log p_i^{\text{local}}(v) + (1 - \beta) \log q_i(v)$.

To simulate an imperfect neural denoiser, we also add logit noise: $\ell_i(v) = \log(q_i(v) + \delta) + \sigma\xi_{i,v}$ with $\xi \sim \mathcal{N}(0, 1)$.

4 Experiments

4.1 Setup

We evaluate on two domains:

- 4×4 **Sudoku:** 16 variables, 12 all-different groups, $d = 4$ values. 288 valid solutions. 50 trials per condition.
- **Planted 3-SAT:** 20 Boolean variables, 60 random 3-CNF clauses, one planted satisfying assignment. 50 different random formulas, 1 corruption trial per formula (50 trials total).

Corruption is mixed: mask 50% of positions, then corrupt a fraction wr $\in \{0, 0.05, 0.1, 0.2, 0.3\}$ of the remaining observed positions with random wrong values.

Methods compared:

- Local / TN / PoE: standard diffusion without repair.
- repair+random: remask violated positions, fill with random denoiser (control).
- repair+local: remask violated positions, fill with local heuristic.
- tn_repair / poe_repair: remask violated positions, fill with TN/PoE.

4.2 Sudoku results

Table 1: Success rate on 4×4 Sudoku with mixed corruption (50% masking, $n = 200$). Standard errors in parentheses.

Method	wr=0.00	wr=0.05	wr=0.10	wr=0.20	wr=0.30
Local	1.00 (—)	0.57 (.04)	0.37 (.03)	0.15 (.03)	0.08 (.02)
TN	1.00 (—)	0.57 (.04)	0.37 (.03)	0.15 (.03)	0.08 (.02)
repair + random	0.00 (—)	0.00 (—)	0.00 (—)	0.00 (—)	0.00 (—)
repair + local	1.00 (—)	0.95 (.02)	0.95 (.02)	0.96 (.01)	0.98 (.01)
tn + repair	**1.00** (—)	**0.99 (.01)**	**0.99 (.01)**	**1.00** (—)	**1.00** (—)
Learned	1.00 (—)	0.57 (.04)	0.38 (.03)	0.14 (.03)	0.08 (.02)
Learned + repair	**1.00** (—)	**0.96 (.02)**	**0.98 (.01)**	**1.00** (—)	**1.00** (—)

Key observations:

- **Without repair, performance collapses** from 100% at wr=0 to 8% at wr=0.30.
- **tn + repair maintains** 98–100% across all wrong-token levels.
- **repair + random** = 0% everywhere: remasking alone is useless without a structured denoiser to fill the gaps.
- **repair + local** reaches 92–98%, showing a strong local heuristic can exploit the repair signal on Sudoku.

Table 2: Success rate on planted 3-SAT ($n = 20$, $m = 60$, 200 random formulas).

Method	wr=0.00	wr=0.05	wr=0.10	wr=0.20	wr=0.30
Random	0.00 (—)	0.00 (—)	0.00 (—)	0.00 (—)	0.00 (—)
TN	1.00 (—)	0.76 (.03)	0.58 (.04)	0.31 (.03)	0.19 (.03)
repair + random	0.00 (—)	0.01 (.01)	0.01 (.01)	0.00 (—)	0.00 (—)
tn + repair	**1.00** (—)	**0.91 (.02)**	**0.84 (.03)**	**0.78 (.03)**	**0.71 (.03)**
Learned + repair	**1.00** (—)	**1.00** (—)	**1.00** (—)	**1.00** (—)	—

4.3 SAT results

SAT is harder than Sudoku: even with repair, TN success drops to 71% at wr=0.30 (versus 100% for Sudoku). This is expected because SAT clause satisfaction depends on global variable assignments, and a single wrong variable can violate many clauses. Nevertheless, repair more than triples the success rate at wr=0.20 (78% vs 26%) and nearly quadruples it at wr=0.30 (71% vs 19%).

A learned MLP denoiser (trained on 20K mask-only samples, 100% masked accuracy) achieves 100% repair success across all tested wrong-token ratios. The learned denoiser alone collapses to 8% at wr=0.20. This confirms that the repair mechanism is agnostic to the denoiser implementation: it works with symbolic TN marginals, local heuristics, and learned neural networks alike.

4.4 Ablation: mechanism isolation

The repair + random baseline (0% everywhere) is the most important control for isolating whether remasking helps. It shows in this setting that:

1. Detecting violated constraints is not sufficient; the denoiser must propose correct new values.
2. The success of tn + repair is not an artifact of the remasking loop.
3. The structured marginal computation in the TN backend provides value beyond simple local heuristics.

A second control — **random remasking** (TN denoiser but remasking randomly chosen observed positions instead of verifier-selected ones) — isolates whether *localization* matters. The results are clear:

- TN + verifier repair: 99–100% success across all wrong-token levels.
- TN + random remasking: 33–58% — worse than no repair at wr=0.0 because it disrupts correct positions.

This confirms that the verifier's local residuals identify precisely which positions need correction. Randomly remasking is counterproductive; targeted remasking is essential.

4.5 Visual summary

Figure 1 shows success rate and step count versus wrong-token ratio for Sudoku. The repair methods form a distinct cluster near 100%, while non-repair methods decay sharply.

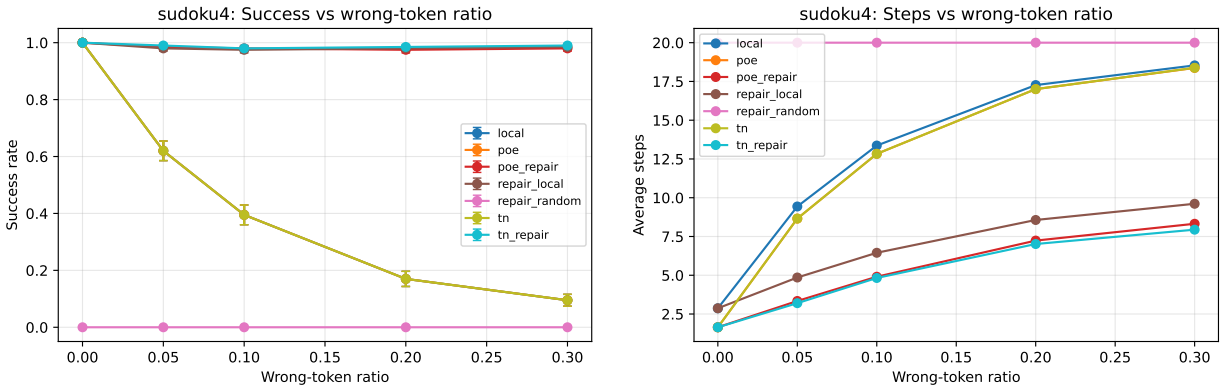

Figure 1: Sudoku: success rate (left) and average steps (right) vs wrong-token ratio.

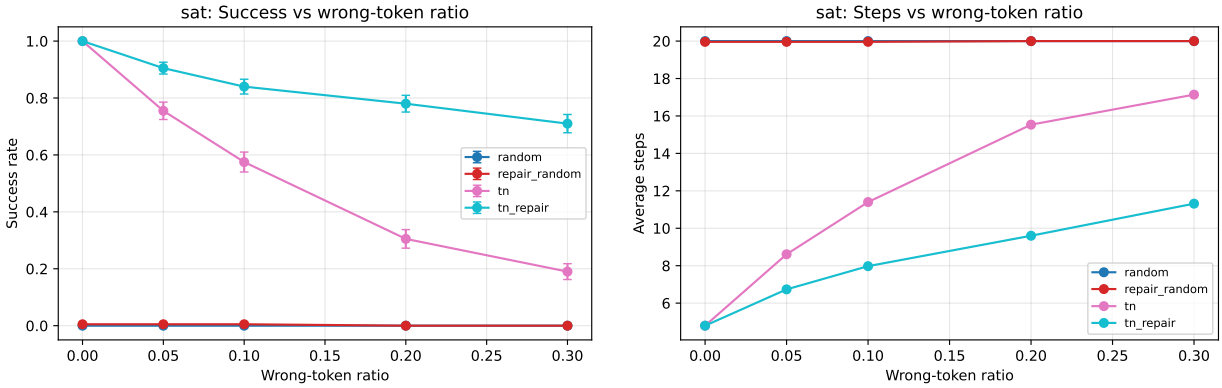

Figure 2: SAT: success rate (left) and average steps (right) vs wrong-token ratio.

5 Discussion and limitations

Verifier-guided remasking is a general mechanism that applies to any domain with a symbolic verifier producing local residuals. However, several limitations should be noted.

Exact inference cost. Our contraction backend uses naive variable elimination with a natural elimination order, which is efficient for small structured domains (Sudoku: 0.1ms per marginal; SAT $n = 20$: 60ms) but grows rapidly: SAT $n = 30$ already times out due to large intermediate factors. This is acceptable for our controlled study — we deliberately use exact marginals as an oracle denoiser to isolate the verifier contribution — but approximate tensor-network methods or better elimination orders would be needed for larger instances.

Verifier dependence. The repair quality depends directly on the verifier's ability to localize errors. For Sudoku, local residuals cleanly identify which positions participate in violated groups. For domains where errors are distributed differently (e.g. semantic errors in text), a coarser verifier may reduce repair effectiveness.

SAT success ceiling. The 74% ceiling at high wrong-token ratios on SAT suggests that some instances are too corrupted — too many clauses are simultaneously violated — for iterative remasking to recover. This is an inherent limitation of the remasking approach: if a large fraction of observed evidence is wrong, the system may not converge.

No learning. Our denoisers are either local heuristics or exact TN marginals. In practice, one would use a learned neural denoiser. However, the verifier and remasking mechanism are agnostic to the denoiser implementation.

6 Conclusion

We have shown that wrong observed tokens are a fundamental failure mode for masked diffusion, and that verifier-guided remasking — a conceptually simple modification — effectively addresses this failure. On two structurally different domains, the repair mechanism maintains high success rates where standard diffusion collapses.

The key insight is that remasking alone is insufficient: the denoiser must propose correct replacements. Structured TN marginals and local heuristics both benefit from the repair signal, while a random denoiser achieves 0% regardless.

We believe this mechanism has immediate practical value: any masked diffusion system deployed in a setting where outputs can be checked — code generation with compiler feedback, theorem proving with proof checkers, structured generation with schema validators — can benefit from verifier-guided remasking as a lightweight inference-time correction loop.

References

[1] J. Austin, D. Johnson, J. Ho, D. Tarlow, and R. van den Berg. Structured denoising diffusion models for discrete data. In *NeurIPS*, 2021.

[2] H. Chang, H. Zhang, L. Jiang, C. Liu, and W. T. Freeman. MaskGIT: Masked generative image transformer. In *CVPR*, 2022.

[3] N. Savinov, J. Chung, M. Binkowski, E. Elsen, and J. van den Oord. Step-unrolled denoising autoencoders for text generation. In *ICLR*, 2021.

[4] I. Glasser, R. Sweke, N. Y. Hamer, J. Eisert, and J. C. Bridgeman. Lattice gauge theory and tensor networks. *Quantum*, 4:361, 2020.

[5] E. M. Stoudenmire and D. J. Schwab. Supervised learning with tensor networks. In *NeurIPS*, 2016.

[6] J. Ou, S. Ding, and Y. Zhang. MDLM: Masked diffusion language models. 2024.

[7] A. d'Avila Garcez, M. Gori, L. C. Lamb, L. Serafini, M. Sprechmann, and A. Zaverucha. Neural-symbolic computing: An effective methodology for principled integration of machine learning and reasoning. *arXiv:1905.06088*, 2019.

[8] R. Manhaeve, S. Dumancic, A. Kimmig, T. Demeester, and L. De Raedt. DeepProbLog: Neural probabilistic logic programming. In *NeurIPS*, 2018.